Classroom Attention Monitoring

Name:
Adharsh - 23PD02
Akilesh S - 23PD33

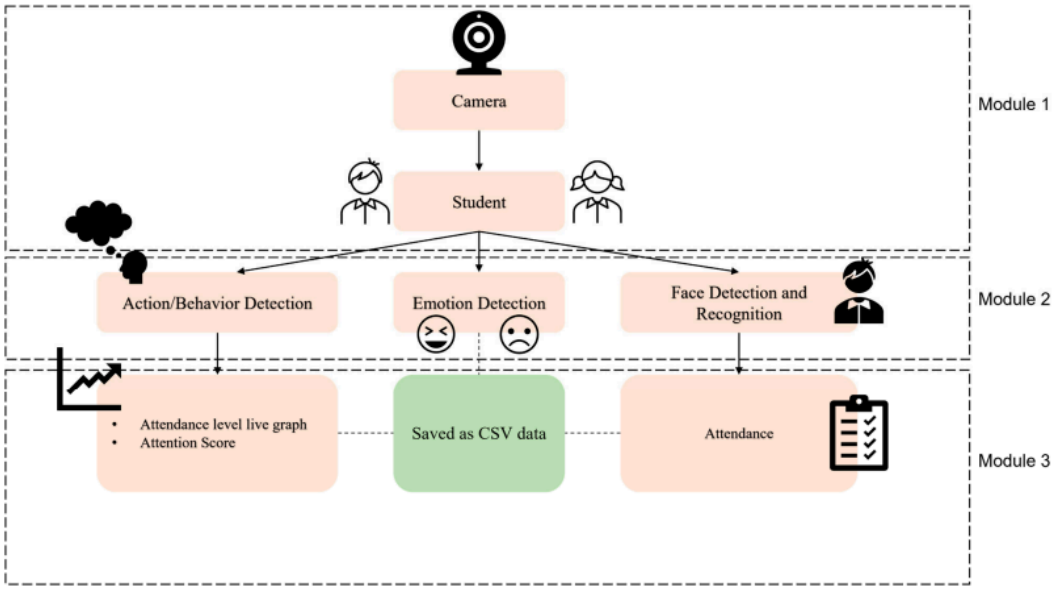
Abstract

The paper presents a real-time, vision-based classroom monitoring system that uses deep learning to assess student attention, emotions, and attendance even when students wear face masks. The system was trained on nine behavioral categories using YOLOv5 model variants. It aims to help instructors adapt their teaching by providing live behavioral and emotional feedback.

Introduction

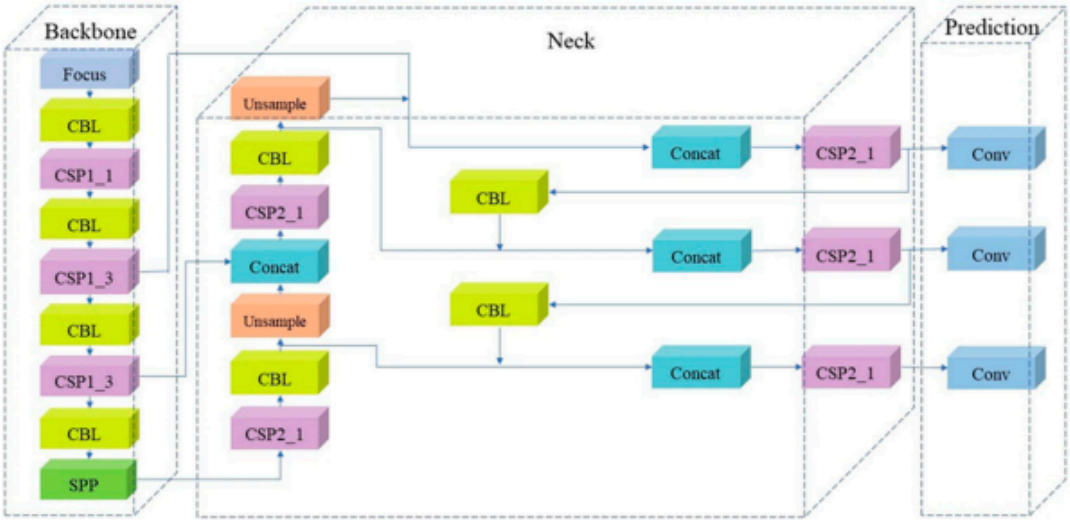
Student engagement is central to effective learning, yet instructors in large classrooms struggle to track individual attention. Existing approaches such as EEG headbands, wearable PPG sensors, and body pose estimation are either intrusive, expensive, or limited in scope. Traditional ML methods also lack real-time capability. The paper identifies these gaps and proposes an AI-powered vision system using YOLOv5 to monitor behavior, emotion, and attendance simultaneously without requiring any wearable device.

Workflow:

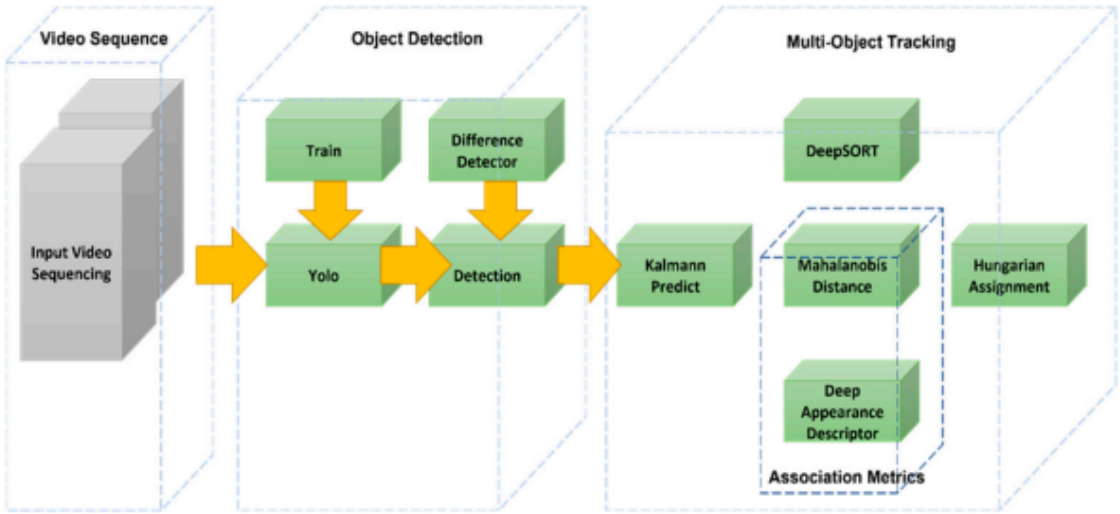


Architecture:

YOLOv5:



DeepSORT:



Methodology

Defines the 7 student behavior classes derived from the SCB-05 dataset (an update from the paper's 9 categories because the paper doesn't provide dataset due to privacy reasons, so we had our own classroom dataset from Kaggle). The classes are:

HIGH attention: blackboard_screen, blackboard_teacher, handrise_read_write, talk_teacher

MEDIUM attention: discussing

LOW attention: standing, talking

Dataset: SCB-05 Dataset

EmotionConfig maps five emotion classes (angry, happy, neutral, sad, surprise) and provides lookup dictionaries that normalize both **AffectNet's** different label schemes into this common set.

YOLOTrainConfig holds all training hyperparameters: image size 512×512, batch size 2, 30 epochs, confidence threshold 0.45, IoU threshold 0.5, and augmentation settings (HSV jitter, horizontal flip at 10%, 10% grayscaling).

DeepSORTConfig configures the tracker: max track age 30 frames, 3 frames to confirm a new track, MobileNet as the appearance embedder, and GPU embedding enabled.

AttentionConfig defines the 3-tier scoring: +1.5 for HIGH, +0.5 for MEDIUM, −1.5 for LOW, clamped to [0, 100], with an initial score of 50 and a rolling smoothing window of 30 frames.AttendanceConfig sets LBPH face recognizer parameters and the Haar Cascade detection settings (scale factor 1.3, min neighbors 5, min face size 60×60, recognition threshold 80.0).

Data Preparation Pipeline

download_datasets.py uses the Kaggle CLI to download three datasets automatically: SCB-05 (student behavior), AffectNet in YOLO format (emotions). It also auto-downloads the OpenCV Haar Cascade XML from GitHub. If the Kaggle CLI is unavailable, it prints the manual download URLs.

prepare_actions_dataset.py reads the raw SCB-05 folder structure, maps folder names to class indices via ActionConfig.scb05_folder_to_class, converts bounding box annotations to YOLO format (normalized x_center y_center width height), and splits data into train/val/test sets.

prepare_emotions_dataset.py does the same for both AffectNet and FER2013, normalizing the 8-class AffectNet scheme and the 7-class FER2013 scheme into the 5-class target label set using the EmotionConfig mapping dictionaries.

- **Train:** 500 images and labels in each class
- **Val:** 125 images and labels in each class
- **Test:** 75 images and labels in each class

Model Training

train_actions.py trains all five **YOLOv5 variants (n, s, m, l, x**) in sequence by calling the YOLOv5 train.py subprocess. It clones the YOLOv5 GitHub repo on first run if it isn't present. Each training run is saved under **outputs/training/actions_{variant}_{timestamp}/** with the standard YOLOv5 output structure (weights, results.csv, confusion matrix, PR curves). After all variants train, it reads their results.csv files and prints a comparison table.

train_emotions.py does the same but trains only the **YOLOv5s** variant on the action dataset, since the paper established **YOLOv5s** as the production choice.

Multi-Object Tracking:

This module wraps the deep-sort-realtime library. **DeepSORT** Tracker converts YOLO-format detections ([x1, y1, w, h], confidence, class) into the format expected by DeepSORT, calls update_tracks(), and converts the results back into a clean list of dicts with track_id, bbox, class, confidence, and confirmed fields. It also maintains a track_history dictionary mapping every track ID to its full per-frame behavior log.

A SimpleTracker fallback is included for environments where deep-sort-realtime isn't installed. It uses pure IoU matching with no appearance embeddings and ages out stale tracks after 30 frames. The create_tracker() factory function tries DeepSORT first and silently falls back to SimpleTracker.

Scoring Engine:

AttentionScorer is the scoring system. It maintains a raw_scores dict (cumulative score per track) and a score_history deque (last 30 frames per track) for smoothing. On each frame, it looks up the behavior's attention level from ActionConfig, applies the appropriate delta, and clamps the result to [0, 100]. The get_smoothed_score() method returns the rolling mean of the history buffer.

Key outputs:

- get_all_scores() — current smoothed score per track

- get_class_attention_summary() — class-wide stats (average, min, max, count of HIGH/MEDIUM/LOW students)
- get_score_summary() — a pandas DataFrame with per-student totals, dominant behavior, and attention-level percentages
- save_reports() — writes attention CSV, time-series CSV, and full detection log CSV to the logs directory

It also stores face-recognition names via set_student_name(track_id, name) so the CSV reports use actual names rather than anonymous IDs.

Face Recognition:

Attendance System implements the two-stage face recognition pipeline:

- **Detection using OpenCV's Haar Cascade classifier**. It runs detectMultiScale() with the configured parameters to find face bounding boxes.
- **Recognition using OpenCV's LBPH recognizer**. LBPH is trained from enrolled face images. On each frame it calls recognizer.predict(), if the returned confidence is below the 80.0 threshold, the face is considered recognized.

enroll_student.py captures 30 reference face images per student, saves them to **data/attendance/enrolled_faces/{student_name}/**, and retrains the LBPH model. The system auto-signs recognized students to a CSV attendance sheet.

The Main Pipeline:

On initialization it loads the action YOLOv5 model (custom weights or falls back to pretrained COCO YOLOv5s with a COCO-to-SCB05 class mapping), the emotion YOLOv5 model, DeepSORT tracker, attendance system, and attention scorer.

The process_frame() method is the per-frame pipeline:

- Run YOLOv5 action detection — list of detections
- Pass detections + frame to DeepSORT — confirmed tracked objects with IDs
- Every 5th frame: run emotion detection (crops the top 40% of each bounding box as the face region)
- Every 30th frame: run Haar+LBPH attendance check and link recognized names to track IDs

- Update attention scores for all tracked objects
- Draw color-coded bounding boxes, attention score bars, HUD overlay, and legend onto the frame
- Push all results to SharedState for the dashboard

The color coding uses green for HIGH attention, amber for MEDIUM, and red for LOW — matching the paper's visualization approach.

SharedState is a thread-safe file bridge. It writes seven live files to logs/dashboard/ on every update: live_frame.jpg, live_frame_small.jpg, live_detections.json, live_scores.json, live_status.json, live_behaviors.json, and live_emotions.json. This decouples the monitoring thread from the Streamlit UI completely — no shared memory needed.

Loads the results.csv produced by each training run and builds a comparison table of precision, recall, mAP@0.5, and F1 score across all five YOLOv5 variants.

Streamlit Web Application:

It auto-discovers all completed training runs by scanning outputs/training/ for folders containing results.csv. It has four pages:

Live Monitor: Polls the logs/dashboard/ files every 2 seconds, displays the live annotated camera frame, real-time student attention cards (HIGH/MEDIUM/LOW), a live attention score chart via Plotly, and a behavior distribution bar chart.

Training Results: Lists all discovered training runs, shows the best run highlighted in green, renders the actual confusion matrix PNG, F1 curve, PR curve, and training loss curves from disk, and allows side-by-side model comparison.

Analytics: Loads the attention_timeseries.csv and detections.csv log files and generates historical attention trend charts and behavior frequency analysis.

About: Project description and usage instructions.

Results

Model metrics:

YOLOv5s - actions :

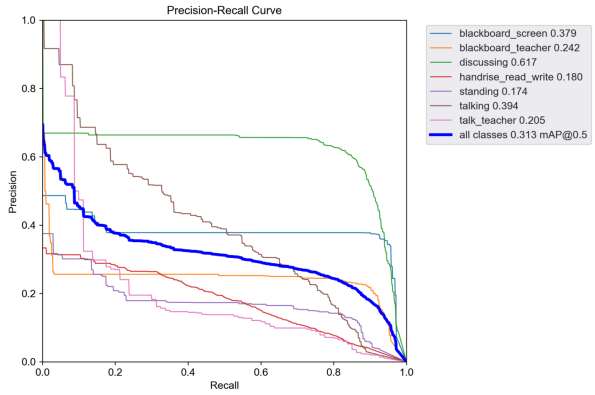
mAP@0.5: 0.3064

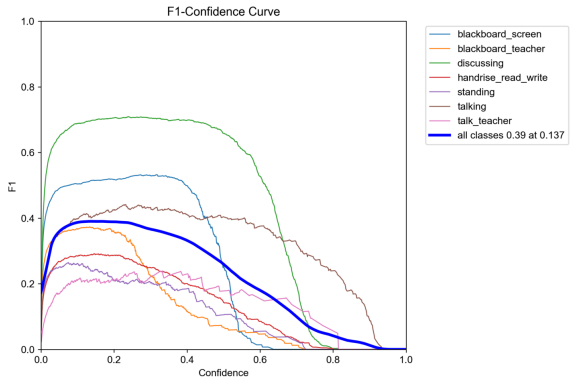
mAP@0.5:0.95: 0.2363

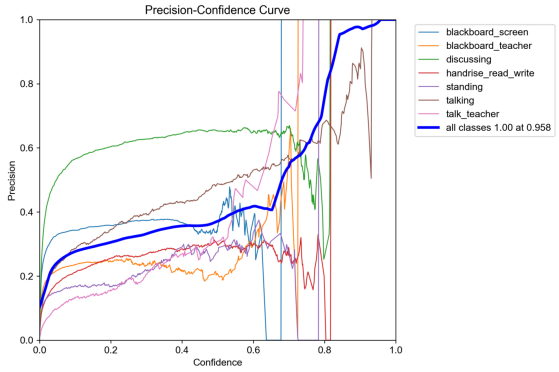
Precision: 0.2825

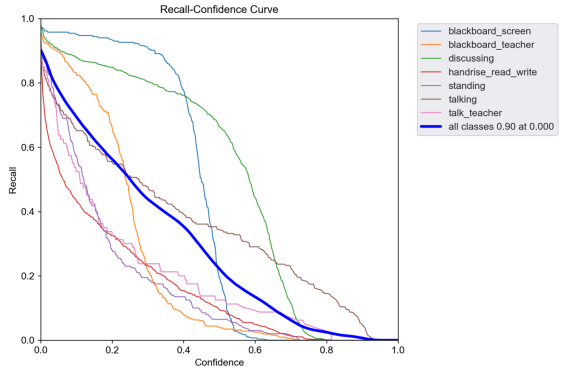
Recall: 0.5507

F1: 0.373

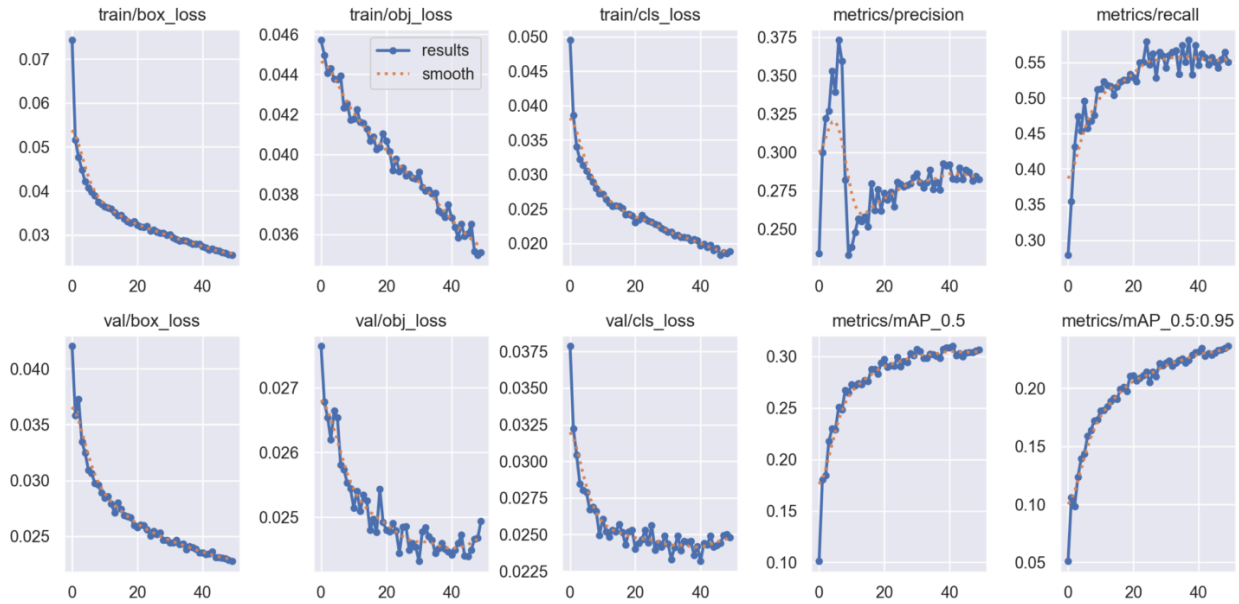






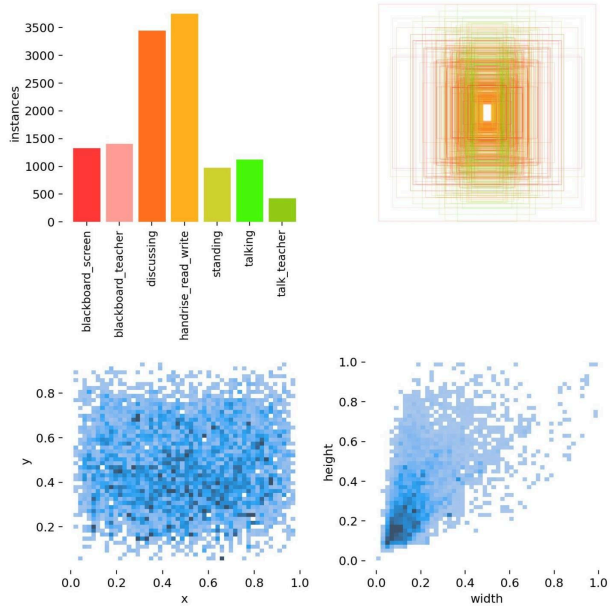


Confusion Matrix
Predicted
True
blackboard_screen
blackboard_teacher
discussing
handrise_read_write
standing
talking
talk_teacher
background
0.60 0.31 0.22
0.28 0.30 0.11 0.03 0.06 0.18
0.80 0.01 0.01 0.02 0.13
0.01 0.20 0.01 0.07 0.01 0.27
0.04 0.15 0.03 0.09 0.06
0.05 0.03 0.13 0.49 0.09
0.03 0.05 0.24 0.04
0.12 0.27 0.18 0.76 0.52 0.37 0.57
0.0 0.1 0.2 0.3 0.4 0.5 0.6 0.7

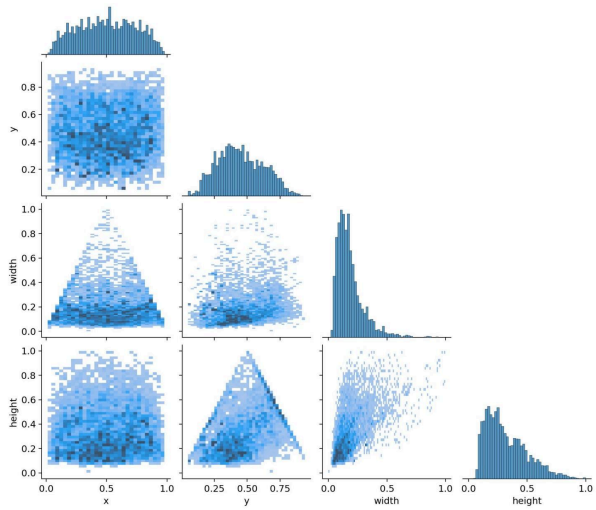

train/box_loss
train/obj_loss
results
smooth
train/cls_loss
metrics/precision
metrics/recall
val/box_loss
val/obj_loss
val/cls_loss
metrics/mAP_0.5
metrics/mAP_0.5:0.95

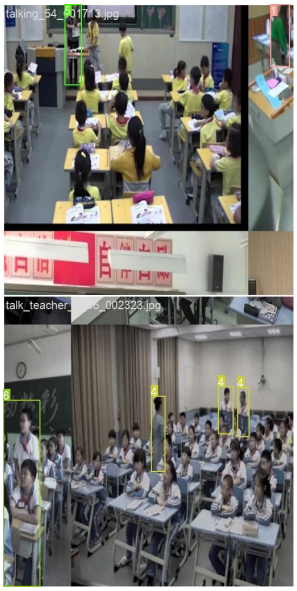
Batch Visualization Samples

Labels:



Label Correlogram:





YOLOv5s - emotions :

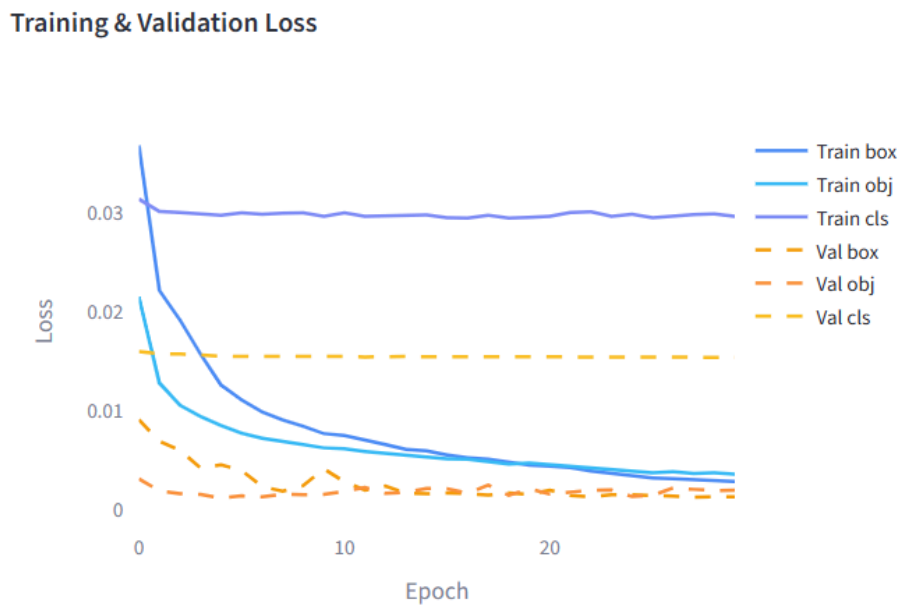
mAP@0.5: 0.2834

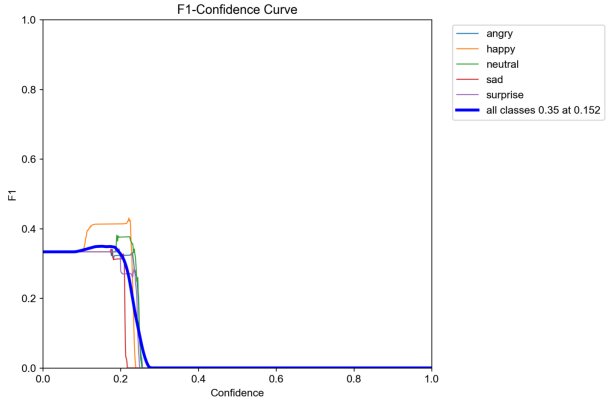
mAP@0.5:0.95: 0.2817

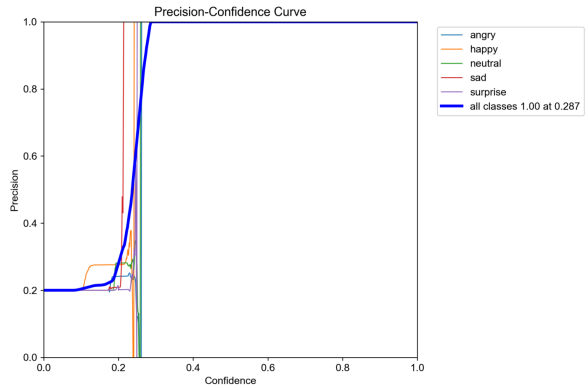
Precision: 0.2151

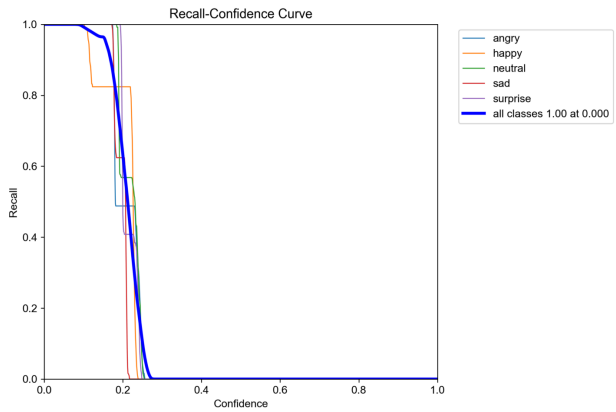
Recall: 0.9648

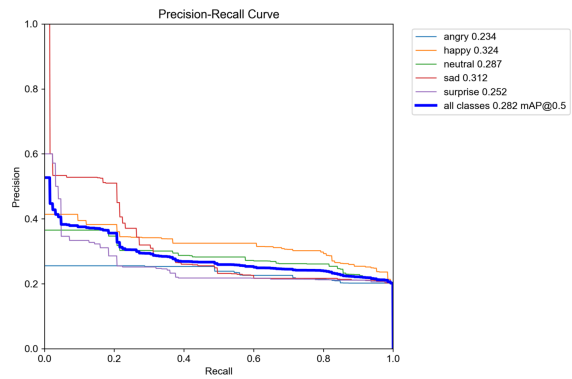
F1: 0.352

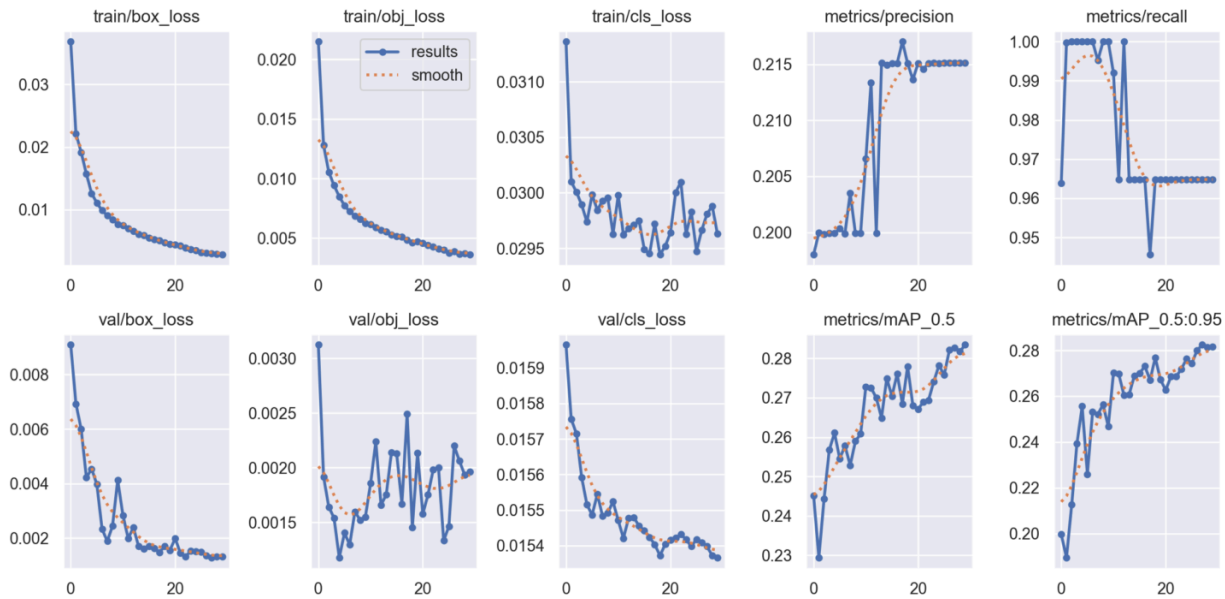


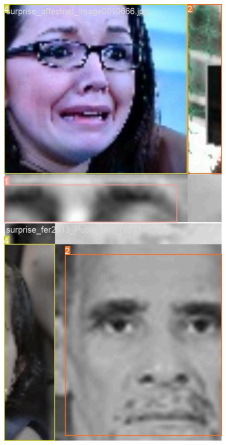


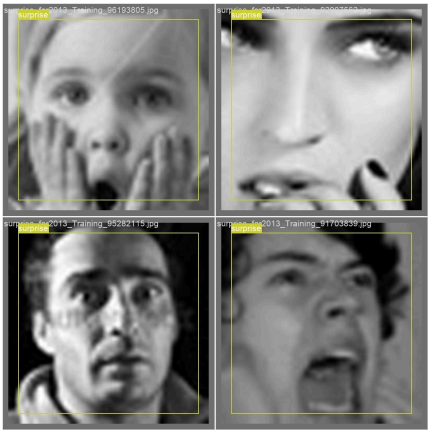






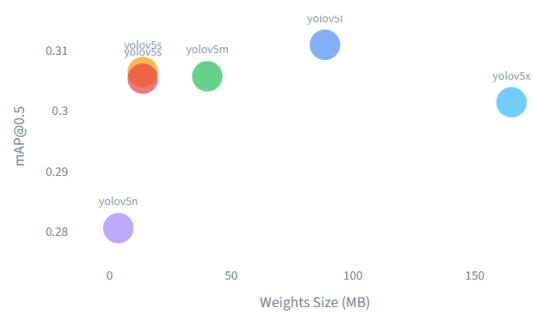






Comparison of all the YOLOv5 variants:

	Run	Variant	mAP50	mAP50 95	Precision	Recall	F1	Epochs	Size MB
0	actions_yolov5l_20260404_0942	yolov5l	0.3109	0.252	0.296	0.6195	0.401	30	88.5
1	actions_yolov5m_20260404_0810	yolov5m	0.3057	0.2444	0.2911	0.5869	0.389	30	40.2
2	actions_yolov5n_20260404_0552	yolov5n	0.2806	0.1977	0.2616	0.5326	0.351	30	3.7
3	actions_yolov5s_20260330_0029	yolov5s	0.3064	0.2363	0.2825	0.5507	0.373	50	13.7
4	actions_yolov5s_20260404_0701	yolov5s	0.3053	0.231	0.2983	0.5418	0.385	30	13.7
5	actions_yolov5x_20260404_1201	yolov5x	0.3014	0.2465	0.2989	0.5695	0.392	30	165.1



Conclusion

The paper concludes that the proposed system successfully monitors student behavior, emotion, and attendance in real time using a non-intrusive, camera-only setup. It outperforms wearable and EEG-based alternatives in practicality and scalability. Key limitations acknowledged include the relatively small training and test datasets. Future work includes integrating the system into social robots that act as teaching assistants, and testing on larger student groups to further improve accuracy.